OKUBE-ENERGY • COURSE PROJECT



Fusing Network and Carbon Cost in a Custom Container Scheduler

Okube: a Go orchestrator that scores every node on latency and on carbon

CUSTOM GO ORCHESTRATOR | NETWORK + CARBON | POWER MODEL FROM SPECPOWER

Sanchit Srisha Udupa (231IT060) **Aditi Pandey (231IT003)**

Guide: Dr. Bhawana Rudra • Department of Information Technology, NIT Karnataka, Surathkal • October 2026

Schedulers are blind to two costs

Network cost

Dependent services placed far apart pay latency on every call, even when each node is well provisioned.

Carbon cost

The same task draws different energy on different nodes, and each node's grid has a different carbon intensity.

- The default Kubernetes scheduler ranks nodes by **resource fit** only.
- A resource-optimal placement is often neither latency-optimal nor carbon-optimal.
- Prior work fixes one cost at a time, so the two goals are never traded off in one decision.

Question. Can one scheduler score nodes on network *and* carbon cost, and cut CO_2 without giving back the latency win?

Each prior work covers one piece

	Network-aware	Carbon signal	Class-aware	Learned power	Fused + ablations
Diktyo (Santos et al.)	✓	×	×	×	×
Piontek et al.	×	✓	✓ (when)	×	×
Saad et al. (Kepler, FL)	×	✓	×	✓	×
Okube-Energy	✓	✓	✓ (where)	✓	✓

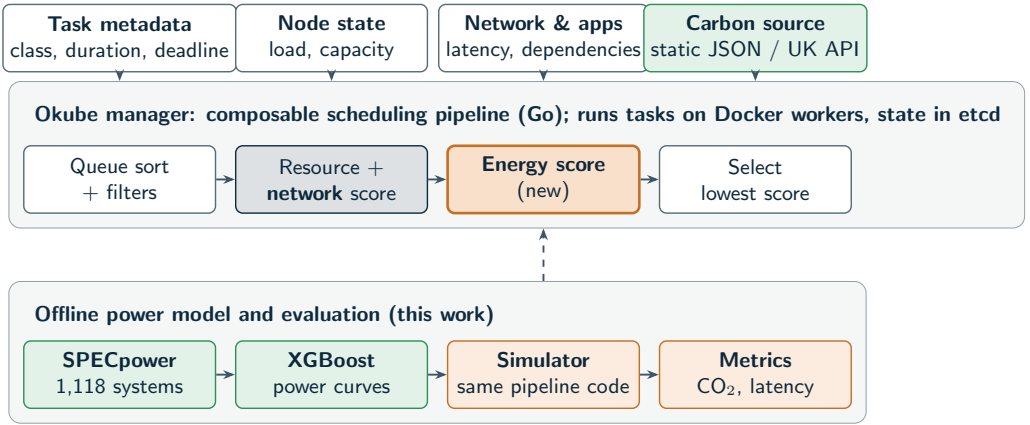
Based on each paper's stated scope.

- **Diktyo**: latency and bandwidth aware placement, carbon-blind.
- **Piontek et al.**: delays shiftable jobs to cleaner windows, no network model, moving-average forecast.
- **Saad et al.**: federated XGBoost energy prediction on Kepler; *stops at prediction*.

What is new in this work

1. **One pipeline, four schedulers.** Network and energy stages toggle independently, so default, network-only, energy-only and fused are ablations of *one* code path. Weight 0 reproduces network-only exactly (tested).
2. **Class-aware fusion.** Carbon weight $\times 0.35$ for critical, $\times 1.4$ for flexible tasks: Piontek's critical/shiftable idea applied to *where*, jointly with network cost.
3. **Carbon cost on a fixed scale.** No min–max stretching of tiny differences; a real $0\,\mathrm{g/kWh}$ grid is a valid value, not "unset".
4. **Power curves learned from 1115 real servers** (SPECpower), replacing a made-up idle/max pair: error 65% $\rightarrow$ 12.0%.
5. **Rigorous evaluation.** Seeded simulator on the real scheduler code; weight chosen on one seed range, reported on a disjoint one; paired, with 95% CIs.

Architecture



The fused score

Network cost (Diktyo-style): summed latency to the nodes hosting a service's dependencies.

Carbon cost on a fixed 0–100 scale:

$$E(n,t) = 100 \, \min\left(1, \tfrac{c_n}{0.3}\right) \, \min\left(1, \tfrac{L_{n,t}}{2}\right)$$

c_n: carbon intensity (gCO_2/kWh /1000); $L_{n,t}$: load multiplier.

Class-aware weight and fused score:

$$w_e(t) = w \times \{0.35,\ 1,\ 1.4\}, \qquad \text{score}(n) = \text{norm}(\text{base}) + w_e(t)\, E$$

Guarantees (all unit-tested)

- $w = 0 \Rightarrow$ network-only, exactly
- zero-carbon grid costs 0
- unknown carbon uses a default
- intensities above the reference saturate

Why a fixed scale: tiny gaps stay tiny

Energy cost assigned to three candidate grids (idle node, small task):

Candidate grids (g/kWh)	Min–max normalised (first version)	Fixed scale (ours)
0, 4, 15 *(all very clean)*	0, 27, 100	**0, 0.7, 2.5**
0, 100, 237 *(a real spread)*	0, 42, 100	**0, 17, 40**

- The first version made a 15 g/kWh node look as bad as possible next to a 0 g/kWh node, so the scheduler would give up latency for a negligible carbon gain.
- On a fixed scale the energy term only matters when the grids genuinely differ.
- Found while testing against real data: the UK API reports exactly **0** for North Scotland, and the old code treated 0 as "unset".

The data we use

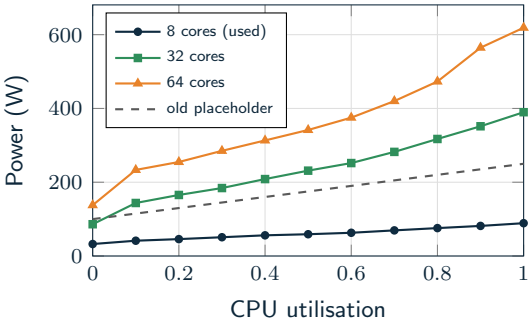
SPECpower_ssj2008 IN RESULTS

- Measured server power at **11 load levels** (idle, 10–100%), published on spec.org.
- **1118 detail pages** scraped, honouring the 10 s crawl delay.
- 1115 systems, 12,265 rows, 2007–2026; multi-node systems divided per node.
- **Use:** XGBoost → power curves → CO_2 in every result.

UK Carbon Intensity API SNAPSHOT

- NESO with Oxford, EDF Europe, WWF; **CC BY 4.0**; no API key.
- Regional gCO_2/kWh for Great Britain, half-hourly, with a forecast.
- Seen 23 Oct 2026: **0** (N. Scotland) to **237** (S. Wales).
- **Use:** `fetch_uk_carbon.py` writes the manager's carbon file (real 0 kept); calibrates the simulated zones.
- **Not yet:** continuous live refresh.

Power model: watts from hardware and load



	Grouped CV		Temporal split	
Model	MAE	MAPE	MAE	MAPE
Placeholder line (100–250 W)	142.5	65.1	250.8	65.9
Line through true idle/max (oracle)	20.5	7.7	33.7	8.2
Ridge regression	75.7	32.9	205.1	46.2
XGBoost (ours)	32.7	12.0	118.0	25.8

- XGBoost, monotone in load; scored on **held-out systems**.
- Placeholder 65% error → model 12.0%.
- A line through true idle/max: 7.7%.
- Newer hardware is harder (26%).

How we evaluate

Simulator. Seeded event loop driving the *real* filter, score and select code; tasks queue when nodes are full.

Cluster. 6 nodes (8 cores) in 3 zones: `scot` 50, `nw` 180, `south` 260 g/kWh (illustrative). The cleanest zone is the farthest from the dirtiest.

Workloads. 60 app deployments, 20/40/40% critical/flexible/default. *Default*: ∼35% utilisation. *Stress*: 2.5× faster arrivals.

Configurations. default, network-only, energy-only, fused.

Metrics. Dynamic CO_2, dependency latency, deadline violations, decision time.

Protocol.

- paired: same trace per seed for every config
- w chosen on seeds 1–30 (largest within +10% latency of network-only), so $w = 1.5$
- reported on *disjoint* seeds 101–150 (50 runs), 95% CIs

DEFAULT WORKLOAD, 50 HELD-OUT SEEDS

Result: less carbon, same latency win

Config	w	CO_2	ΔCO_2 (%)	Lat. (ms)	ΔLat. (%)
Default	–	25.66	–	13.62	–
Network-only	–	25.20	-1.7 ± 0.9	2.26	-83.3
Energy-only	0.2	24.91	-2.8 ± 0.8	11.82	-12.9
Fused	0.2	24.32	-5.2 ± 1.1	2.09	-84.6
Energy-only	1.5	23.77	-7.3 ± 0.8	10.84	-20.2
Fused	1.5	22.34	-12.9 ± 1.2	2.57	-81.0

Δ vs default, paired; $\pm$ is a 95% CI. Rows below the rule use $w = 1.5$.

−12.9%

CO_2 vs default (± 1.2)

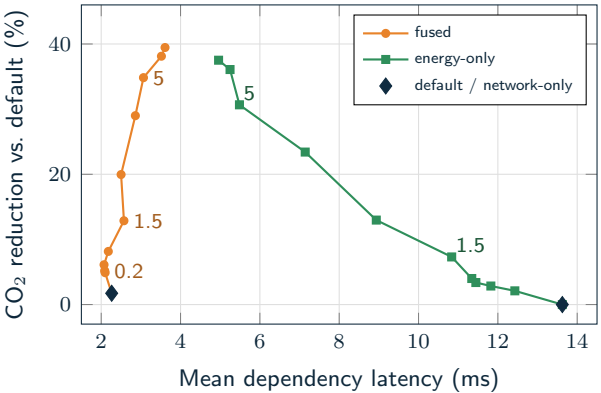
−81%

latency vs default

(network-only: −83%)

Energy-only at the same weight: −7.3% CO_2 but latency only −20%. Under stress the carbon gain shrinks to −2.4%.

The weight trades carbon against latency



- Fused sits far left of energy-only: same carbon, a fraction of the latency.
- Labels give w; we pick $\mathbf{1.5}$ by a 10% latency budget.
- A looser budget picks a larger w (about 35% CO_2 at $w = 5$).

By class (fused, $w = 1.5$): flexible → 119 g/kWh nodes (default 162), critical → 183.

What the results do not show

- **Fused vs. energy-only is not a pure fusion test:** fused down-weights load balancing (0.3), energy-only does not.
- **The learned model only counts CO_2;** placement uses the heuristic score.
- **Carbon is illustrative and static.** Absolute CO_2 is relative, not real emissions; no time-shifting.
- **Simulation only:** topological latency, one scenario family, identical hardware, no live run.
- **One non-federated power model:** 12.0% held-out error, 26% on newer systems.
- **Stress:** gains nearly vanish; deadlines do not separate configs.

FUTURE WORK

Where this goes next

- **Live UK carbon feed** with the static file as offline fallback.
- **Temporal placement**: use the forecast to delay flexible jobs into cleaner windows.
- **Predicted watts in the score** instead of the load heuristic.
- **Federated power model**, as in Saad et al.
- **Fair ablation and a live multi-worker run.**
- **Kubernetes Scheduling Framework port.**

Conclusion

- One pipeline scores nodes on **network and carbon** cost, class-aware and independently switchable.
- A **fixed-scale carbon cost** and a **power model learned from 1115 real servers** make the evaluation honest.
- At $w = 1.5$: $-$**12.9% CO_2** with latency still **81% below default**, which neither network-only nor energy-only reaches.
- Limits are clear: simulation, illustrative carbon, one hardware type, saturated load.

Thank you • Questions?